Part 2. 로봇 AI 수학적 심화

목차

1. 개요
2. 공통 기초 이론
3. RT-1: Robotics Transformer
4. RT-2: VLM을 VLA로 전환
5. OpenVLA: 오픈소스 7B VLA
6. π0: Flow Matching 기반 액션 생성
7. CoT-VLA: 시각적 사고 연쇄
8. 추가 모델들
9. PyTorch 구현 예시
10. 하이퍼파라미터 가이드
11. 참고 문헌

---

1. 개요

이 문서는 **Vision-Language-Action (VLA) 모델**의 수학적 원리와 구현을 심층 분석합니다.

1.1 Part 2에서 다루는 내용

- **수학적 형식화**: 각 모델의 핵심 알고리즘을 수식으로 표현
- **아키텍처 심화**: 레이어별 구조와 파라미터 분석
- **학습 알고리즘**: 손실 함수, 최적화 기법
- **PyTorch 구현**: 실제 코드로 이론 검증

1.2 사전 지식 요구사항

- 선형대수: 행렬 연산, 내적, 고유값 분해
- 확률론: 확률 분포, 조건부 확률, 베이즈 정리
- 딥러닝 기초: 역전파, 경사하강법, Transformer
- PyTorch 기초: `nn.Module`, `forward()`, 자동미분

---

2. 공통 기초 이론

2.1 Behavior Cloning의 수학적 정의

목표: 전문가 데이터 $\mathcal{D} = \{(o_i, a_i)\}_{i=1}^{N}$에서 정책 π_θ를 학습

$$\pi_\theta^* = \arg\min_\theta \mathbb{E}_{(o,a)\sim\mathcal{D}}[\mathcal{L}(\pi_\theta(o), a)]$$

손실 함수:

- 연속 행동: $\mathcal{L}_{\text{MSE}} = \|a - \pi_\theta(o)\|_2^2$
- 이산 행동: $\mathcal{L}_{\text{CE}} = -\sum_k a_k \log \pi_\theta(o)_k$

한계:

- **Distributional Shift**: 학습 분포와 실행 분포 불일치
- **Covariate Shift**: 작은 오차가 누적되어 미지의 상태 도달

2.2 Transformer 기초 복습

Self-Attention 수학

입력 시퀀스 $X \in \mathbb{R}^{N\times d}$에 대해:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

직관:

- Query: "나는 무엇을 찾는가?"
- Key: "나는 무엇을 제공하는가?"
- Value: "내가 제공하는 실제 정보"

Multi-Head Attention

$$\text{MHA}(X) = \text{Concat}(h_1, \ldots, h_H)W_O$$

$$h_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$$

장점: 여러 표현 부분공간을 병렬로 학습

2.3 비전 인코더 개요

인코더	학습 방법	강점
EfficientNet	Supervised (ImageNet)	효율성, 파라미터 효율적 스케일링
ViT (Vision Transformer)	Supervised/Self-supervised	장거리 의존성, 스케일 잘 됨
DINO-v2	Self-supervised (Knowledge Distillation)	공간 일관성, 세그멘테이션

SigLIP	Contrastive (Language-Image)	언어 정렬, 시맨틱 이해
CLIP	Contrastive (Language-Image)	제로샷 인식, 다양한 개념

3. RT-1: Robotics Transformer

3.1 전체 아키텍처 개요

```
입력: [6장의 이미지 (300×300×3), 텍스트 명령]
  ↓
[텍스트 인코더: Universal Sentence Encoder]
  → 텍스트 임베딩: 512차원
  ↓
[이미지 인코더: FiLM-EfficientNet-B3]
  각 이미지 → 9×9×1536 피처맵
  ↓
[Token Learner]
  9×9×1536 → 8개 토큰 (512차원)
  ↓
[6장 × 8토큰 = 48개 토큰 시퀀스]
  ↓
[Transformer Decoder (8 layers)]
  Causal masking으로 시간적 일관성 유지
  ↓
[Action Heads]
  11개 행동 차원 각각 256-bin 분류
  ↓
출력: (Δx, Δy, Δz, Δroll, Δpitch, Δyaw, gripper) ∈ ℝ⁷
     + (x, y, z, terminate) ∈ {0,1}⁴
```

3.2 EfficientNet-B3 백본

Compound Scaling

EfficientNet의 핵심: 깊이(depth), 너비(width), 해상도(resolution)를 동시에 스케일

$$\text{depth} : d = \alpha^\phi$$

$$\text{width} : w = \beta^\phi$$

$$\text{resolution} : r = \gamma^\phi$$

제약: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \alpha \geq 1, \beta \geq 1, \gamma \geq 1$

B3의 경우: $\phi = 2 \rightarrow (d, w, r) \approx (1.4, 1.2, 1.15)$

Mobile Inverted Bottleneck Convolution (MBConv)

```
Input: x ∈ ℝ^{H×W×C}
  ↓
1×1 Conv (Expansion, C → C*t)
  ↓
Depthwise 3×3 or 5×5
  ↓
Squeeze-and-Excitation (SE)
  ↓
1×1 Conv (Projection, C*t → C')
  ↓ Residual Add (if stride=1 and C=C')
Output: y ∈ ℝ^{H/s × W/s × C'}
```

Squeeze-and-Excitation (SE) 모듈

채널별 중요도를 adaptive하게 조절:

$$z = \mathrm{GlobalAvgPool}(x) \in \mathbb{R}^C$$

$$s = \sigma(W_2 \cdot \mathrm{ReLU}(W_1 \cdot z)) \in \mathbb{R}^C$$

$$\hat{x} = x \odot s$$

여기서 $W_1 \in \mathbb{R}^{C/r \times C}$, $W_2 \in \mathbb{R}^{C \times C/r}$, r은 축소 비율 (보통 4).

EfficientNet-B3 구조

Stage	Operator	Resolution	Channels	Layers
1	Conv 3×3	300×300	32	1
2	MBConv1, k3	150×150	16	1
3	MBConv6, k3	75×75	24	2
4	MBConv6, k5	38×38	40	2
5	MBConv6, k3	19×19	80	3
6	MBConv6, k5	10×10	112	3
7	MBConv6, k5	10×10	192	4
8	MBConv6, k3	9×9	320	1
9	Conv 1×1, Pool	9×9	1536	1

최종 출력: $9 \times 9 \times 1536$ 피처맵

3.3 FiLM — 조건부 특징 변조

FiLM의 수학적 정의

FiLM (Feature-wise Linear Modulation)은 조건 c (텍스트)에 따라 특징을 변조:

$$\text{FiLM}(x \mid c) = \gamma(c) \odot x + \beta(c)$$

여기서:

- $x \in \mathbb{R}^{H \times W \times C}$: 피처맵
- $c \in \mathbb{R}^{d_c}$: 조건 임베딩
- $\gamma : \mathbb{R}^{d_c} \to \mathbb{R}^C$: 스케일 함수
- $\beta : \mathbb{R}^{d_c} \to \mathbb{R}^C$: 시프트 함수

다른 조건부 방법과의 비교

방법	수식	특징
Concatenation	$h = f([x; c])$	조건을 입력에 붙임
Addition	$h = f(x + g(c))$	조건을 더함
FiLM	$h = \gamma(c) \odot f(x) + \beta(c)$	조건이 특징을 **변조**

FiLM의 표현력

ReLU 활성화와 결합 시:

$$\text{ReLU}(\gamma \cdot x + \beta) = \begin{cases} \gamma x + \beta & \text{if } \gamma x + \beta > 0 \\ 0 & \text{otherwise} \end{cases}$$

γ, β를 조절하여:

- ReLU의 "켜짐/꺼짐" 영역 제어
- 뉴런의 기울기(gain) 조절
- 태스크별 특화된 표현 학습

RT1에서의 FiLM 구현

```python
class FiLMLayer(nn.Module):
    def __init__(self, num_features, conditioning_dim=512):
        super().__init__()
        self.gamma_fc = nn.Linear(conditioning_dim, num_features)
        self.beta_fc = nn.Linear(conditioning_dim, num_features)

        # 초기화: gamma=1, beta=0 (항등 변환으로 시작)
        nn.init.ones_(self.gamma_fc.bias)
        nn.init.zeros_(self.gamma_fc.weight)
        nn.init.zeros_(self.beta_fc.bias)
        nn.init.zeros_(self.beta_fc.weight)

    def forward(self, x, conditioning):
        # x: (B, C, H, W)
        # conditioning: (B, conditioning_dim)
        gamma = self.gamma_fc(conditioning)  # (B, C)
        beta = self.beta_fc(conditioning)    # (B, C)

        gamma = gamma[:, :, None, None]  # (B, C, 1, 1)
        beta = beta[:, :, None, None]    # (B, C, 1, 1)

        return gamma * x + beta
```

3.4 Token Learner — 적응적 토큰 선택

동기: 왜 모든 공간 위치가 필요하지 않은가?

로봇 조작에서 중요한 영역:

- **그리퍼 근처**: 현재 상태
- **목표 물체**: 조작 대상
- **장애물**: 회피 필요
- **작업대/배경**: 기준점

$9 \times 9 = 81$개 공간 위치 중 대부분은 중복 정보.

Token Learner의 수학적 형식화

입력: $F \in \mathbb{R}^{H \times W \times C}$ (피처맵) **출력**: $Z \in \mathbb{R}^{S \times C}$ (S개의 토큰, RT1에서 $S = 8$)

Step 1: 공간 어텐션 가중치 계산

각 출력 토큰 $s \in \{1, \ldots, S\}$에 대해:

$$\alpha_s = \text{softmax}(\text{MLP}_s(F)) \in \mathbb{R}^{H \times W}$$

MLP_s: 각 공간 위치 (h, w)의 피처 $F_{h,w} \in \mathbb{R}^C$를 스칼라로 매핑

$$\text{MLP}_s(F_{h,w}) = W_s^{(2)} \cdot \text{GELU}(W_s^{(1)} \cdot F_{h,w})$$

$W_s^{(1)} \in \mathbb{R}^{C/4 \times C}, W_s^{(2)} \in \mathbb{R}^{1 \times C/4}$

Step 2: 가중 평균으로 토큰 생성

$$z_s = \sum_{h,w} \alpha_s[h,w] \cdot F_{h,w} \in \mathbb{R}^C$$

Step 3: 프로젝션 (optional)

$$\tilde{z}_s = W_{\text{proj}} \cdot z_s \in \mathbb{R}^D$$

RT1에서 $C = 1536 \rightarrow D = 512$

PyTorch 구현

```
class TokenLearner(nn.Module):
    def __init__(self, in_channels, num_tokens=8, bottleneck_dim=64
        super().__init__()
        self.num_tokens = num_tokens

        # 각 토큰에 대한 공간 어텐션 네트워크
        self.attention_mlps = nn.ModuleList([
            nn.Sequential(
                nn.Linear(in_channels, bottleneck_dim),
                nn.GELU(),
                nn.Linear(bottleneck_dim, 1)
            )
            for _ in range(num_tokens)
        ])

    def forward(self, x):
        # x: (B, C, H, W)
        B, C, H, W = x.shape
        x_flat = x.permute(0, 2, 3, 1).reshape(B, H*W, C)  # (B, H*

        tokens = []
        for mlp in self.attention_mlps:
            # 각 위치의 중요도 스코어
            scores = mlp(x_flat)  # (B, H*W, 1)
            attn = F.softmax(scores, dim=1)  # (B, H*W, 1)

            # 가중 평균
            token = (attn * x_flat).sum(dim=1)  # (B, C)
            tokens.append(token)

        return torch.stack(tokens, dim=1)  # (B, num_tokens, C)
```

3.5 RT1의 Transformer Decoder

입력 시퀀스 구성

6장의 이미지에서 각 8개 토큰 → 48개 토큰:

$$Z = [z_1^{(1)}, \ldots, z_8^{(1)}, z_1^{(2)}, \ldots, z_8^{(2)}, \ldots, z_1^{(6)}, \ldots, z_8^{(6)}]$$

$$Z \in \mathbb{R}^{48 \times 512}$$

위치 인코딩

이미지 내 위치 + **시간적 위치**를 구분하는 2D 위치 인코딩:

$$\mathrm{PE}[i, j] = \mathrm{PE}_{\text{spatial}}[j \mod 8] + \mathrm{PE}_{\text{temporal}}[i//8]$$

Causal Masking

RT1은 시간 축에 대해 causal masking 적용:

$$M[i, j] = \begin{cases} 0 & \text{if } \lfloor i/8 \rfloor \geq \lfloor j/8 \rfloor \\ -\infty & \text{otherwise} \end{cases}$$

이미지 t의 토큰들은 이미지 $1, \ldots, t$의 토큰만 참조.

이유:

- 추론 시 KV-cache 사용 가능
- 새 이미지가 들어오면 이전 이미지의 KV를 재계산하지 않음
- 실시간 제어 효율성

3.6 RT1 손실 함수

다중 헤드 Cross-Entropy

11개 행동 차원을 **독립적**으로 예측:

$$\mathcal{L}_{\text{RT1}} = \sum_{k=1}^{11} \mathcal{L}_{\text{CE}}^{(k)}$$

각 차원의 cross-entropy:

$$\mathcal{L}_{\text{CE}}^{(k)} = -\sum_{b=0}^{255} y_b^{(k)} \log p_b^{(k)}$$

여기서:

- $y^{(k)}$: 정답의 원-핫 인코딩
- $p^{(k)} = \mathrm{softmax}(\mathrm{logit}^{(k)})$: 모델 예측

행동 이산화 (Action Discretization)

연속 행동 $a \in [-1, 1]$을 256개 bin으로 양자화:

$$\text{bin}(a) = \text{clip}\left(\left\lfloor \frac{(a+1) \cdot 256}{2} \right\rfloor, 0, 255\right)$$

역변환:

$$a \approx \frac{2 \cdot \text{bin}}{256} - 1$$

독립 예측의 한계

문제: 행동 차원 간 **상관관계** 포착 못함

예:

- 대각선 이동 ($\Delta x > 0, \Delta y > 0$ 동시)
- 물체 잡기 (z 하강 + gripper close)

해결책:

- **Autoregressive 예측**: $p(a) = \prod_k p(a_k \mid a_{<k})$ (RT2, OpenVLA)
- **연속 공간 생성**: Flow Matching (π0)
- **Diffusion**: DDPM (Octo)

---

4. RT-2: VLM을 VLA로 전환

4.1 사전학습된 VLM의 활용

RT2는 **PaLM-E (13B)** 또는 **PaLI-X (55B)**를 백본으로 사용.

PaLM-E 아키텍처

```
이미지 → ViT 인코더 → 이미지 토큰 (256개)
    ↓
[이미지 토큰 | 텍스트 토큰] → PaLM Decoder → 다음 토큰 예측
```

핵심: 이미지 토큰과 텍스트 토큰이 **동일한 임베딩 공간**에서 처리됨.

RT2의 확장

PaLM-E에 **행동 토큰**을 추가:

```
[이미지 토큰 | 텍스트 프롬프트 | 행동 토큰]
```

행동 토큰은 텍스트 토큰과 동일한 어휘에서 선택.

4.2 Shared Embedding Space

임베딩 행렬의 구조

$$E \in \mathbb{R}^{V \times D}$$

$V = 32{,}000$ (전체 어휘), D = 모델 차원

$$E = \begin{pmatrix} E_{\text{common}} \\ E_{\text{action}} \end{pmatrix} = \begin{pmatrix} E_{1:31744} \\ E_{31745:32000} \end{pmatrix}$$

초기 상태: E_{action}은 원래 거의 사용되지 않던 토큰의 임베딩.

파인튜닝 후: E_{action}이 행동의 의미를 담도록 재학습됨.

출력 레이어에서의 처리

출력 logit:

$$\text{logit} = hW_{\text{output}}^{\top} \in \mathbb{R}^{V}$$

추론 시 행동 토큰만 유효하게:

$$\text{logit}_i = \begin{cases} \text{logit}_i & \text{if } i \in [31745, 32000] \\ -\infty & \text{otherwise} \end{cases}$$

$$p = \text{softmax}(\text{logit})$$

자연어 토큰의 확률이 0이 되어, 모델이 행동 토큰만 출력.

4.3 Loss Masking

학습 시 시퀀스:

$$[v_1, \ldots, v_{256}, w_1, \ldots, w_M, a_1^*, \ldots, a_8^*]$$

- v: 이미지 패치 토큰
- w: 텍스트 프롬프트 토큰
- a^*: 정답 행동 토큰

Next-token prediction:

위치 i에서 토큰 t_i가 주어졌을 때 다음 토큰 t_{i+1} 예측.

문제: 이미지/텍스트 위치에서의 "다음 토큰"은 의미가 없음.

해결: Loss Masking

$$\mathcal{L} = -\sum_{i \in \mathcal{I}_{\text{action}}} \log p(a_i^* \mid t_{<i})$$

$\mathcal{I}_{\text{action}}$: 행동 토큰 위치만 포함

```
def compute_loss(logits, labels, action_mask):
    """
    logits: (B, L, V)
    labels: (B, L)
    action_mask: (B, L) - 행동 위치만 1, 나머지 0
    """
    loss = F.cross_entropy(logits.view(-1, V), labels.view(-1), redu
    loss = loss.view(B, L)
    loss = (loss * action_mask).sum() / action_mask.sum()
    return loss
```

4.4 Vision Encoder Freezing

RT2는 ViT를 **동결(frozen)**한 채 사용.

장점:

1. **Catastrophic Forgetting 방지**: 웹 스케일 지식 보존
2. **계산 효율**: ViT 그래디언트 계산 불필요
3. **이론적 근거**: 로봇 이미지 분포 ⊂ 웹 이미지 분포

반대 의견 (OpenVLA):

충분히 큰 로봇 데이터(100만+ 트라젝토리)가 있으면 unfrozen > frozen

4.5 RT2의 학습 프로세스

```
# 의사코드
for batch in dataloader:
    images, text, actions = batch

    # 1. 이미지 인코딩 (frozen)
    with torch.no_grad():
        image_tokens = vit_encoder(images)

    # 2. 텍스트 토큰화
    text_tokens = tokenizer(text)

    # 3. 행동 토큰화
    action_tokens = action_tokenizer(actions)

    # 4. 시퀀스 구성
    input_seq = concat([image_tokens, text_tokens, action_tokens[:-1
    target_seq = concat([pad, pad, action_tokens])

    # 5. 마스크 생성
    action_mask = (target_seq != pad)

    # 6. Forward
    logits = transformer_decoder(input_seq)

    # 7. Loss (행동 토큰에만)
    loss = masked_cross_entropy(logits, target_seq, action_mask)

    # 8. Backward
    loss.backward()
    optimizer.step()
```

---

5. OpenVLA: 오픈소스 7B VLA

5.1 Prismatic VLM 백본

OpenVLA는 **Prismatic** VLM을 기반으로:

```
이미지 → DINO-v2 인코더 → 공간 특징 (1024차원)
      ↘ SigLIP 인코더 → 시맨틱 특징 (768차원)
          ↓ Concatenate (1792차원)
          ↓ MLP Projector
          ↓ 256개 비전 토큰 (4096차원)
          ↓ Llama 2 7B Decoder
```

5.2 DINO-v2 — 자기 지도 학습

학습 목표

Student-Teacher 프레임워크:

같은 이미지의 다른 view x, x'에 대해:

$$\mathcal{L} = -\sum_i p^{(i)}_{\text{teacher}}(x') \log p^{(i)}_{\text{student}}(x)$$

Centering과 Sharpening:

Teacher 출력에 centering 적용 (mode collapse 방지):

$$p_{\text{teacher}} = \text{softmax}\left(\frac{g_{\text{teacher}}(x') - c}{\tau_t}\right)$$

c는 지수이동평균:

$$c \leftarrow m \cdot c + (1-m) \cdot \mathbb{E}[g_{\text{teacher}}(x')]$$

Student는 더 높은 온도:

$$p_{\text{student}} = \text{softmax}\left(\frac{g_{\text{student}}(x)}{\tau_s}\right)$$

$\tau_t < \tau_s$로 설정하여 teacher가 더 sharp.

EMA 업데이트:

$$\theta_{\text{teacher}} \leftarrow \lambda\theta_{\text{teacher}} + (1-\lambda)\theta_{\text{student}}$$

$\lambda = 0.996 \to 1$ 코사인 스케줄.

DINO-v2의 특징

- **공간적 일관성**: 같은 물체의 다른 부분이 유사한 특징
- **세그멘테이션 창발**: 어텐션 맵이 물체 경계와 일치
- **로봇 활용**: 물체 위치, 그리퍼-물체 관계 파악에 강점

5.3 SigLIP — 언어-이미지 정렬

CLIP vs SigLIP

CLIP (Contrastive):

배치 내 모든 이미지-텍스트 쌍에 대해:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^{B}\exp(s_{ij}/\tau)}$$

$s_{ij} = f_I(I_i)^\top f_T(T_j)$: 유사도

문제: Softmax 분모가 배치 전체 → 배치 크기에 민감

SigLIP (Sigmoid):

각 쌍을 **독립적**으로 이진 분류:

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B^2}\sum_{i,j}\left[y_{ij}\log\sigma(s_{ij}-b) + (1-y_{ij})\log(1-\sigma(s_{ij}-b))\right]$$

$y_{ij} = 1$ if (I_i, T_j) 매칭, else 0 b: 학습 가능한 바이어스

장점:

- 계산 효율 (softmax 분모 불필요)
- 배치 크기 독립적
- 빠른 수렴

5.4 듀얼 인코더 결합

채널 방향 Concatenation

$$f^{(i)}_{\text{dual}} = [f^{(i)}_{\text{DINO}} \parallel f^{(i)}_{\text{SigLIP}}] \in \mathbb{R}^{1792}$$

각 패치 i에서 두 인코더 출력을 concatenate.

MLP Projector

$$h^{(i)} = W_2 \cdot \text{GELU}(W_1 \cdot f^{(i)}_{\text{dual}} + b_1) + b_2$$

$W_1 \in \mathbb{R}^{d_h \times 1792}$, $W_2 \in \mathbb{R}^{4096 \times d_h}$

결과: 256개의 4096차원 비전 토큰 → Llama 2 입력

왜 Concatenation?

실험적 비교 (Prismatic 논문):

결합 방법	VQA 정확도
DINO only	64.2
SigLIP only	68.1
Addition	69.3
Concatenation	**71.5**
Cross-Attention	70.8

Concatenation이 최고 성능:

- 두 표현의 정보 손실 없음
- MLP가 결합 방법 학습

5.5 Open X-Embodiment 데이터셋

구성

- 33+ 기관
- 22+ 로봇 유형
- 60+ 데이터셋
- 100만+ 트라젝토리
- 5억+ 프레임

데이터 형식 통일

```
{
    "observation": {
        "image": np.array,  # (H, W, 3) uint8
        "state": np.array,  # (D_state,) 로봇 고유감각
    },
    "action": np.array,     # (D_action,) Task Space
    "language_instruction": str,
}
```

Task Space 통일:

모든 로봇의 행동을 7차원으로 정규화:

$$a = (\Delta x, \Delta y, \Delta z, \Delta\phi, \Delta\theta, \Delta\psi, g) \in [-1, 1]^7$$

5.6 LoRA — 효율적 파인튜닝

Low-Rank Adaptation

가중치 행렬 $W_0 \in \mathbb{R}^{d\times k}$를 업데이트하는 대신:

$$W = W_0 + \Delta W = W_0 + BA$$

$B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, $r \ll \min(d, k)$

왜 Low-Rank?

가설: 파인튜닝의 가중치 변화 ΔW는 low-rank 구조.

실험적으로, ΔW의 상위 소수 특이값이 대부분 정보 담음.

초기화

$$A \sim \mathcal{N}(0, \sigma^2), \quad B = 0$$

초기에 $\Delta W = BA = 0$ → 원본 모델과 동일하게 시작.

OpenVLA 적용

```
lora_target_modules = [
    "q_proj", "k_proj", "v_proj", "o_proj",  # Attention
    "gate_proj", "up_proj", "down_proj"       # MLP
]

lora_r = 32        # rank
lora_alpha = 64    # scaling factor
scaling = lora_alpha / lora_r  # = 2.0

# Forward
h = W_0 @ x + scaling * (B @ A) @ x
```

파라미터 효율성

7B 모델:

- 전체 파라미터: ~7B
- LoRA 파라미터 (r=32): ~80M
- **비율: 1.1%**

---

6. π0: Flow Matching 기반 액션 생성

6.1 Flow Matching vs Diffusion

확률 흐름

Diffusion: 확률적 미분방정식 (SDE)

$$dx_t = f(x_t, t)dt + g(t)dW_t$$

dW_t: 브라운 운동

Flow Matching: 상미분방정식 (ODE)

$$\frac{dx_t}{dt} = v_\theta(x_t, t)$$

노이즈 항 없음 → **결정적 경로**

경로의 기하학

```
Diffusion:  x₀ ~~~~→ x_T (구불구불, 확률적)
Flow:       x₀ ----→ x_T (직선, 결정적)
```

직선 경로 → 더 적은 스텝으로 이동 가능.

6.2 Conditional Flow Matching — 완전 유도

목표

데이터 분포 $p_1(x)$에서 노이즈 분포 $p_0(x) = \mathcal{N}(0, I)$로의 흐름 학습.

조건부 흐름

$x_1 \sim p_1$ (데이터)가 주어졌을 때:

$$x_t = (1-t)x_0 + tx_1, \quad x_0 \sim \mathcal{N}(0, I)$$

속도 벡터:

$$\frac{dx_t}{dt} = x_1 - x_0$$

학습 목표

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x_0,x_1}\left[\|v_\theta(x_t, t) - (x_1 - x_0)\|_2^2\right]$$

여기서:

- $t \sim \text{Uniform}[0, 1]$
- $x_0 \sim \mathcal{N}(0, I)$
- $x_1 \sim p_{\text{data}}$
- $x_t = (1-t)x_0 + tx_1$

Diffusion과의 관계

Diffusion (ϵ-prediction):

$$\mathcal{L}_{\text{Diff}} = \mathbb{E}_{t,\epsilon,x_0}\left[\|\epsilon_\theta(x_t, t) - \epsilon\|_2^2\right]$$

유사한 형태이나 차이는 **경로 형태**와 **스케줄**.

6.3 π0의 Action Expert 아키텍처

```
[VLM 백본: PaLI Gemma 3]
이미지 (2-3 views) + 텍스트 → VLM Prefix Tokens (KV 캐시)

[Action Expert]
입력: [VLM Prefix | Robot State | Noisy Action]
출력: Velocity Vector (noise → action 방향)
```

Block-wise Attention

전체 시퀀스를 3개 블록으로:

$$X = [X_{\text{VLM}} \mid X_{\text{state}} \mid X_{\text{action}}]$$

어텐션 마스크:

$$M = \begin{pmatrix} \mathbf{1}_{N_V \times N_V} & \mathbf{0} & \mathbf{0} \\ \mathbf{1}_{N_S \times N_V} & \mathbf{1}_{N_S \times N_S} & \mathbf{0} \\ \mathbf{1}_{N_A \times N_V} & \mathbf{1}_{N_A \times N_S} & \mathbf{1}_{N_A \times N_A} \end{pmatrix}$$

정보 흐름: VLM → State → Action (단방향)

로봇별 프로젝션 헤드

Action Expert 출력: $h \in \mathbb{R}^{H \times D}$

로봇 r에 대한 프로젝션:

$$A_r = W_r h + b_r, \quad W_r \in \mathbb{R}^{n_r \times D}$$

n_r: 로봇 r의 자유도

6.4 Flow Matching 샘플링 — ODE 적분

Euler Method

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t, c)$$

```python
def euler_sample(v_theta, condition, steps=10):
    x = torch.randn(shape)  # x_0 ~ N(0, I)
    dt = 1.0 / steps

    for i in range(steps):
        t = i * dt
        v = v_theta(x, t, condition)
        x = x + dt * v

    return x  # x_1 (action)
```

고차 적분 (Heun's Method)

$$k_1 = v_\theta(x_t, t, c)$$

$$k_2 = v_\theta(x_t + \Delta t \cdot k_1, t + \Delta t, c)$$

$$x_{t+\Delta t} = x_t + \frac{\Delta t}{2}(k_1 + k_2)$$

실용적 선택: Euler 8-10 스텝이 충분.

6.5 π0의 학습 알고리즘

```
def train_step(batch, model, optimizer):
    images, text, robot_state, action_gt = batch

    # VLM forward → prefix tokens
    prefix = model.vlm_encode(images, text)

    # Flow matching
    t = torch.rand(batch_size)  # t ~ U[0,1]
    noise = torch.randn_like(action_gt)  # x_0 ~ N(0,I)
    x_t = (1 - t[:, None, None]) * noise + t[:, None, None] * actio

    # Action expert forward
    v_pred = model.action_expert(prefix, robot_state, x_t, t)

    # Loss
    v_gt = action_gt - noise
    loss = F.mse_loss(v_pred, v_gt)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

---

7. CoT-VLA: 시각적 사고 연쇄

7.1 서브골 이미지 예측

동기

가설: 행동의 **결과**를 예측하면 행동의 **품질**이 향상.

- "이 행동을 하면 어떤 장면이 보일까?"
- 시각적 그라운딩으로 물리적 타당성 학습
- 암묵적 세계 모델 학습

LLM CoT와의 비유

LLM CoT	CoT-VLA
문제 → 추론 스텝 → 답	현재 이미지 → 서브골 이미지 → 행동

7.2 RQVAE — 이미지의 이산 표현

VQ-VAE 복습

인코더 출력 $z = E(I)$를 코드북 $\mathcal{C} = \{c_1, \ldots, c_K\}$에서 양자화:

$$\hat{z} = \text{Quantize}(z) = c_k, \quad k = \arg\min_i \|z - c_i\|_2$$

Residual Quantization

VQ-VAE를 Q단 적용:

```
z = E(I)
for q = 1 to Q:
    ẑ^(q) = Quantize_q(r^(q-1))
    r^(q) = r^(q-1) - ẑ^(q)
r^(0) = z
최종: ẑ = Σ_q ẑ^(q)
```

이미지 → 토큰 시퀀스

공간 $H' \times W'$, Q단 RVQ:

토큰 수: $H' \times W' \times Q$

예: $H' = W' = 16$, $Q = 8$ → 2048 토큰

7.3 듀얼 손실 함수

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{visual}} + \lambda \mathcal{L}_{\text{action}}$$

$\mathcal{L}_{\text{visual}}$: **서브골 이미지 예측**

$$\mathcal{L}_{\text{visual}} = -\sum_{i=1}^{N_{\text{img}}} \log p_\theta(c_i^* \mid c_{<i}^*, I_t, l)$$

c_i^*: 미래 이미지 I_{t+k}의 i번째 RQVAE 토큰

$\mathcal{L}_{\text{action}}$: **행동 예측**

$$\mathcal{L}_{\text{action}} = -\sum_{j=1}^{H \cdot d_a} \log p_\theta(a_j^* \mid \hat{I}_{t+k}, I_t, l)$$

핵심: 행동 예측이 서브골 이미지에 의존.

7.4 액션-프리 비디오 활용

행동 라벨 없는 비디오도 활용:

$$\mathcal{L}_{\text{video}} = -\sum_i \log p_\theta(c_i^{(t+k)} \mid c_{<i}^{(t+k)}, I_t)$$

효과:

- 로봇 데이터 없이도 시각적 동역학 학습
- 서브골 예측 품질 향상

---

8. 추가 모델들

8.1 Octo (Berkeley, 2024)

특징

- **크기**: 93M (Small), 360M (Base)
- **데이터**: Open X-Embodiment
- **출력**: Diffusion 기반 액션

아키텍처

```
[Transformer 인코더]
이미지 + 텍스트 + 로봇 상태 → Task Embedding

[Diffusion 디코더]
Task Embedding 조건으로 액션 청크 생성
```

Diffusion Head:

$$\mathcal{L}_{\text{Octo}} = \mathbb{E}_{t,\epsilon}\left[\|\epsilon - \epsilon_\theta(a_t, t, c)\|_2^2\right]$$

Octo vs OpenVLA

특성	Octo	OpenVLA
크기	93M / 360M	7B

행동 생성	Diffusion	Autoregressive
제로샷 성능	낮음	높음
파인튜닝	빠름	느림 (LoRA)
추론 속도	느림	빠름

8.2 RT-X (Google, 2023)

RT-1/RT-2를 Open X-Embodiment로 학습.

핵심 발견

1. **스케일링 효과**: 더 많은 데이터/로봇 → 더 나은 성능
2. **전이 학습**: 로봇 A 데이터가 로봇 B 성능 향상
3. **Positive Transfer**: 대부분의 조합에서 효과

8.3 GR00T N1.6 (NVIDIA, 2025)

이중 시스템 아키텍처

```
[Slow Brain: Cosmos Reason]
고수준 추론, 계획 수립

[Fast Body: GR00T Foundation]
저수준 제어, 실시간 모터 명령
```

전신 제어

$$a_t = (\underbrace{\theta_1, \ldots, \theta_7}_{\text{왼팔}}, \underbrace{\theta_8, \ldots, \theta_{14}}_{\text{오른팔}}, \underbrace{\theta_{15}, \ldots, \theta_{26}}_{\text{양다리}}, \ldots)$$

31+ DOF 관절 각도 동시 제어.

균형 제약:

$$\mathrm{CoM}_{xy} \in \mathrm{ConvexHull}(\text{contact points})$$

---

9. PyTorch 구현 예시

9.1 FiLM 조건부 CNN

```python
import torch
import torch.nn as nn
from torchvision.models import efficientnet_b3

class FiLMEfficientNet(nn.Module):
    def __init__(self, text_dim=512, num_film_layers=5):
        super().__init__()
        self.backbone = efficientnet_b3(pretrained=True)

        film_channels = [40, 80, 112, 192, 320]
        self.film_layers = nn.ModuleList([
            FiLMLayer(c, text_dim) for c in film_channels[:num_film_
        ])

        self.film_positions = [2, 3, 4, 5, 6]

    def forward(self, x, text_embedding):
        features = x
        film_idx = 0

        for i, layer in enumerate(self.backbone.features):
            features = layer(features)

            if i in self.film_positions and film_idx < len(self.filı
                features = self.film_layers[film_idx](features, texı
                film_idx += 1

        return features

class FiLMLayer(nn.Module):
    def __init__(self, num_features, conditioning_dim):
        super().__init__()
        self.gamma_fc = nn.Linear(conditioning_dim, num_features)
        self.beta_fc = nn.Linear(conditioning_dim, num_features)

        nn.init.ones_(self.gamma_fc.bias)
        nn.init.zeros_(self.gamma_fc.weight)
        nn.init.zeros_(self.beta_fc.bias)
        nn.init.zeros_(self.beta_fc.weight)

    def forward(self, x, cond):
        gamma = self.gamma_fc(cond).unsqueeze(-1).unsqueeze(-1)
        beta = self.beta_fc(cond).unsqueeze(-1).unsqueeze(-1)
        return gamma * x + beta
```

9.2 Token Learner

```python
class TokenLearner(nn.Module):
    def __init__(self, in_channels, num_tokens=8, bottleneck_dim=64
        super().__init__()
        self.num_tokens = num_tokens

        self.spatial_attn = nn.Sequential(
            nn.Conv2d(in_channels, bottleneck_dim, 1),
            nn.GELU(),
            nn.Conv2d(bottleneck_dim, num_tokens, 1)
        )

    def forward(self, x):
        B, C, H, W = x.shape

        attn_logits = self.spatial_attn(x)  # (B, num_tokens, H, W)
        attn = F.softmax(attn_logits.view(B, self.num_tokens, -1), 

        x_flat = x.view(B, C, -1)  # (B, C, H*W)
        tokens = torch.bmm(attn, x_flat.transpose(1, 2))

        return tokens
```

9.3 RT1 전체 모델

```python
class RT1(nn.Module):
    def __init__(self, text_dim=512, num_images=6, num_tokens=8,
                 hidden_dim=512, num_layers=8):
        super().__init__()

        self.text_encoder = nn.Linear(768, text_dim)
        self.vision_encoder = FiLMEfficientNet(text_dim)
        self.token_learner = TokenLearner(1536, num_tokens)
        self.proj = nn.Linear(1536, hidden_dim)

        self.pos_embed = nn.Parameter(
            torch.randn(1, num_images * num_tokens, hidden_dim)
        )

        decoder_layer = nn.TransformerDecoderLayer(
            d_model=hidden_dim, nhead=8, dim_feedforward=2048,
            dropout=0.1, activation='gelu', batch_first=True
        )
        self.transformer = nn.TransformerDecoder(decoder_layer, num_

        self.action_heads = nn.ModuleList([
            nn.Linear(hidden_dim, 256) for _ in range(11)
        ])

    def forward(self, images, text_embedding):
        B = images.shape[0]
        text_feat = self.text_encoder(text_embedding)

        all_tokens = []
        for i in range(6):
            feat = self.vision_encoder(images[:, i], text_feat)
            tokens = self.token_learner(feat)
            all_tokens.append(tokens)

        tokens = torch.cat(all_tokens, dim=1)
        tokens = self.proj(tokens) + self.pos_embed

        out = self.transformer(tokens, tokens)
        pooled = out.mean(dim=1)

        action_logits = [head(pooled) for head in self.action_heads
        return action_logits
```

9.4 Flow Matching Action Expert

```python
class FlowMatchingActionExpert(nn.Module):
    def __init__(self, vlm_dim=2048, state_dim=14, action_dim=14,
                 chunk_size=50, hidden_dim=512, num_layers=6):
        super().__init__()

        self.chunk_size = chunk_size
        self.action_dim = action_dim

        self.state_encoder = nn.Sequential(
            nn.Linear(state_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, hidden_dim)
        )

        self.action_encoder = nn.Sequential(
            nn.Linear(action_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, hidden_dim)
        )

        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.GELU()
        )

        self.vlm_proj = nn.Linear(vlm_dim, hidden_dim)

        self.transformer = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(
                d_model=hidden_dim, nhead=8,
                dim_feedforward=2048, batch_first=True
            ),
            num_layers=num_layers
        )

        self.velocity_head = nn.Linear(hidden_dim, action_dim)

    def forward(self, vlm_prefix, robot_state, noisy_action, t):
        vlm_tokens = self.vlm_proj(vlm_prefix)
        state_token = self.state_encoder(robot_state).unsqueeze(1)

        action_tokens = self.action_encoder(noisy_action)
        time_emb = self.time_embed(t).unsqueeze(1)
        action_tokens = action_tokens + time_emb

        tokens = torch.cat([vlm_tokens, state_token, action_tokens],
        out = self.transformer(tokens)

        action_out = out[:, -self.chunk_size:]
        velocity = self.velocity_head(action_out)
```

```
        return velocity
```

9.5 Flow Matching 학습/추론

```
def train_flow_matching(model, dataloader, optimizer):
    model.train()

    for batch in dataloader:
        vlm_prefix, robot_state, action_gt = batch
        B = action_gt.shape[0]

        t = torch.rand(B, device=action_gt.device)
        noise = torch.randn_like(action_gt)

        t_expand = t[:, None, None]
        noisy_action = (1 - t_expand) * noise + t_expand * action_gt

        v_pred = model(vlm_prefix, robot_state, noisy_action, t)
        v_gt = action_gt - noise

        loss = F.mse_loss(v_pred, v_gt)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

@torch.no_grad()
def sample_flow_matching(model, vlm_prefix, robot_state, steps=10):
    model.eval()
    B = vlm_prefix.shape[0]

    action = torch.randn(
        B, model.chunk_size, model.action_dim,
        device=vlm_prefix.device
    )

    dt = 1.0 / steps
    for i in range(steps):
        t = torch.full((B,), i * dt, device=action.device)
        v = model(vlm_prefix, robot_state, action, t)
        action = action + dt * v

    return action
```

---

10. 하이퍼파라미터 가이드

10.1 RT1/RT2 스타일

하이퍼파라미터	권장값	설명
Learning Rate	1e-4 ~ 3e-4	AdamW
Batch Size	64 ~ 256	GPU 메모리에 따라
Image Size	224 ~ 300	EfficientNet: 300
Num Images	2 ~ 6	더 많으면 문맥 향상
Token Learner Tokens	8	RT1 기본값
Transformer Layers	6 ~ 12	규모에 따라
Hidden Dim	512 ~ 1024	규모에 따라

10.2 OpenVLA 스타일

하이퍼파라미터	권장값	설명
Learning Rate	2e-5	VLM 기준 낮게
Epochs	20 ~ 30	BC는 많은 epoch
Warmup Steps	1000	Linear warmup
LR Schedule	Cosine	Annealing
LoRA Rank	16 ~ 64	메모리/성능
LoRA Alpha	32 ~ 128	rank의 2배

10.3 π0 스타일

하이퍼파라미터	권장값	설명
Learning Rate	1e-4	Flow matching
Chunk Size	50	1~2.5초
ODE Steps	8 ~ 16	품질/속도
ODE Solver	Euler	단순하고 효과적

---

11. 참고 문헌

1. **RT-1**: Brohan et al., "RT-1: Robotics Transformer for Real-World Control at Scale", RSS 2023
2. **RT-2**: Brohan et al., "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control", CoRL 2023

3. **OpenVLA**: Kim et al., "OpenVLA: An Open-Source Vision-Language-Action Model", 2024
4. **π0**: Black et al., "π0: A Vision-Language-Action Flow Model for General Robot Control", 2024
5. **CoT-VLA**: Shang et al., "CoT-VLA: Visual Chain-of-Thought Reasoning", 2025
6. **Octo**: Team et al., "Octo: An Open-Source Generalist Robot Policy", 2024
7. **Open X-Embodiment**: "Open X-Embodiment: Robotic Learning Datasets and RT-X Models", 2023
8. **Flow Matching**: Lipman et al., "Flow Matching for Generative Modeling", ICLR 2023
9. **DINO-v2**: Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision", 2023
10. **SigLIP**: Zhai et al., "Sigmoid Loss for Language Image Pre-Training", ICCV 2023

다음 파트: Part 3. 추가 커리큘럼 — 시뮬레이션, 강화학습, 휴머노이드